

Universitatea Tehnica din Cluj-Napoca
Departament Calculatoare

Programming Techniques in Java

Reflection Techniques

T. Cioara, V. Chifu, C. Pop, D. Mitrea
2025-2026

Reflection - Overview

- Ability of a running program to examine itself and change its actions depending on what it finds
- Dynamic access/inspection to internal information for classes

ADVANTAGES

- Building flexible code that can be assembled at run time
- No required source code links between components

DISADVANTAGES

- Security
- Code Maintenance
- Performance
- Issues when debugging reflection code for classes which are not accessible at compile time

Reflection – Motivation Scenario

- Mapping a Java class to the SQL statement for creating a database table according to the class definition

```
class Product {  
    int id;  
    String name;  
    double price;  
    boolean inStock;  
}
```



```
CREATE TABLE products (  
    id INT PRIMARY KEY,  
    name VARCHAR(255),  
    price DOUBLE,  
    in_stock BOOLEAN  
);
```

- **Solution 1:** manually write the SQL statements for creating the table according to the class definition
 - Developer must manually keep SQL in sync with Java
 - Renaming *price* to *unitPrice* breaks everything
 - Different developers may interpret types differently

**The Java model and the database schema
are duplicated sources of truth**

Reflection – Motivation Scenario

- Mapping a Java class to the SQL statement for creating a database table according to the class definition
 - **Solution 2:** automatic mapping using **annotations** and **reflection**

Annotation Type Interfaces

Syntax

```
<modifiers> @ interface <annotation-type-name> {  
    // Annotation type body goes here  
}
```

The same as for an interface declaration (public/package level)

Static and default methods are not allowed here

Role:

- Associates (or annotates) metadata (or notes) to the program elements: package, class, interface, field of a class, local variable, method, etc.) in a Java program
- Acts like a decoration or a note for the program element that it annotates

Usage

```
@annotationType(name1=value1, name2=value2, names3=values3...)
```

Restrictions:

- **R1:** An annotation type cannot inherit from another annotation type
- **R2:** Method declarations in an annotation type cannot specify any parameters
- **R3:** Method declarations in an annotation type cannot have a throws clause
- **R4:** The return type of a method declared in an annotation type must be one of the following types: any primitive type, java.lang.String, java.lang.Class, an enum type, an annotation type, an array of any of the previous mentioned type
- **R5:** An annotation type cannot declare a method, which would be equivalent to overriding a method in the Object class or the Annotation interface
- **R6:** An annotation type cannot be generic

Reflection – Motivation Scenario

- Mapping a Java class to the SQL statement for creating a database table according to the class definition
 - **Solution 2:** automatic mapping using **annotations** and **reflection**

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@interface Table {
    String name();
}
```

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
    String type();
    boolean primaryKey() default false;
}
```

```
@Table(name = "products")
class Product {
    @Column(name = "id", type = "INT", primaryKey = true)
    int id;
    @Column(name = "name", type = "VARCHAR(255)")
    String name;
    @Column(name = "price", type = "DOUBLE")
    double price;
    @Column(name = "in_stock", type = "BOOLEAN")
    boolean inStock;
}
```

Reflection – Motivation Scenario

- Mapping a Java class to the SQL statement for creating a database table according to the class definition
 - **Solution 2:** automatic mapping using **annotations** and **reflection**

```
public static String generateCreateTable(Class<?> clazz) {
    StringBuilder sql = new StringBuilder();
    Table table = clazz.getAnnotation(Table.class);
    sql.append("CREATE TABLE ").append(table.name()).append(" (\n");
    Field[] fields = clazz.getDeclaredFields();
    StringBuilder primaryKey = new StringBuilder();
    for (Field field : fields) {
        Column column = field.getAnnotation(Column.class);
        sql.append("    ").append(column.name()).append(" ").append(column.type());
        if (column.primaryKey()) { primaryKey.append(column.name()); }
        sql.append(",\n");
    }
    if (primaryKey.length() > 0) {
        sql.append(" PRIMARY KEY (").append(primaryKey).append(")\n");
    } else {
        sql.setLength(sql.length() - 2);
        sql.append("\n");
    }
    sql.append(");");
    return sql.toString();
}
```

Class becomes the single source of truth.

SQL is generated automatically.

Works for any class, not just Product.

This is the foundation of ORMs like Hibernate.

Metadata

Metadata is data about data

- Needs to have a representation of itself
- Metadata is organized into objects called meta-objects
- **Introspection** - runtime examination of meta-objects

Metadata for a class is stored in `java.lang.Class`

- This is the entry point into reflection operations

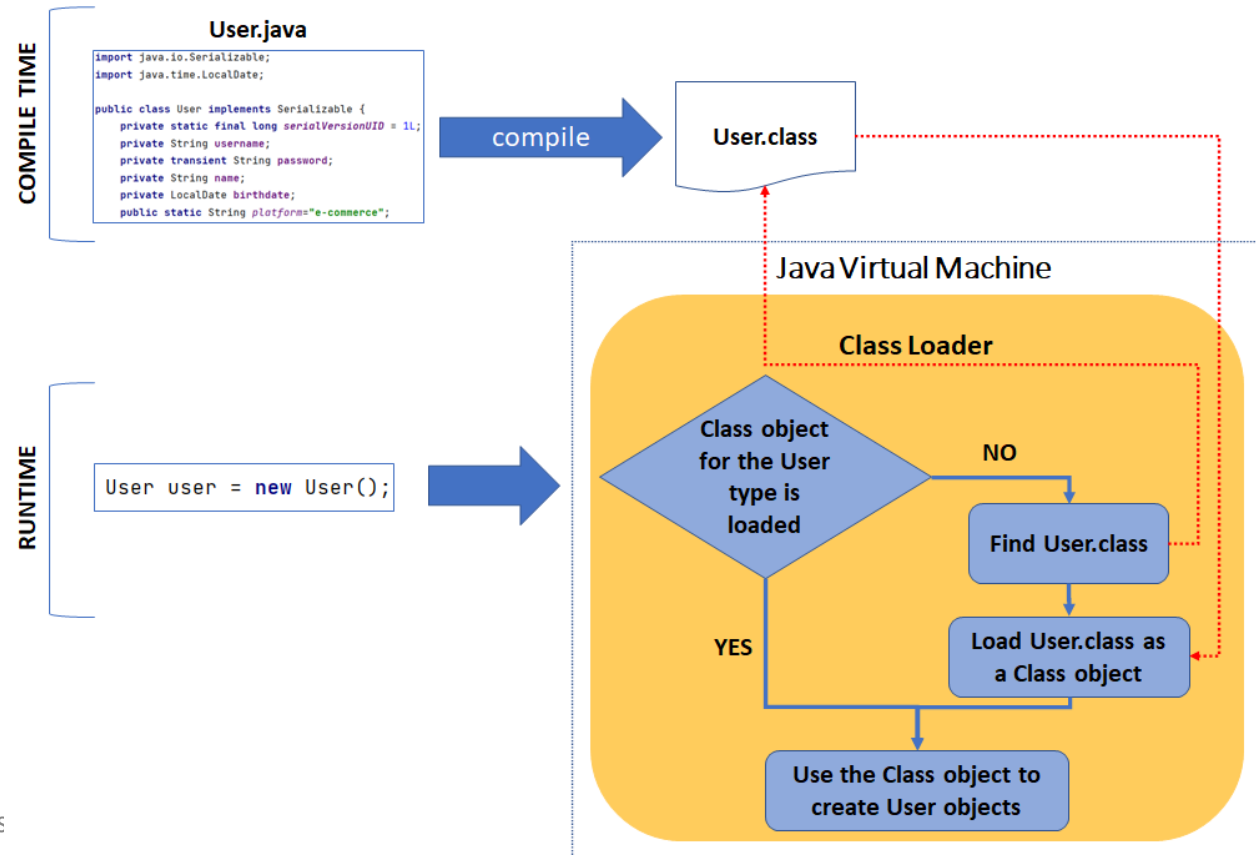
Metadata includes information about

- The class itself, like package and superclass of the class
- The interfaces implemented by the class
- Details of the constructors, fields, and methods defined by the class

Runtime Type Information

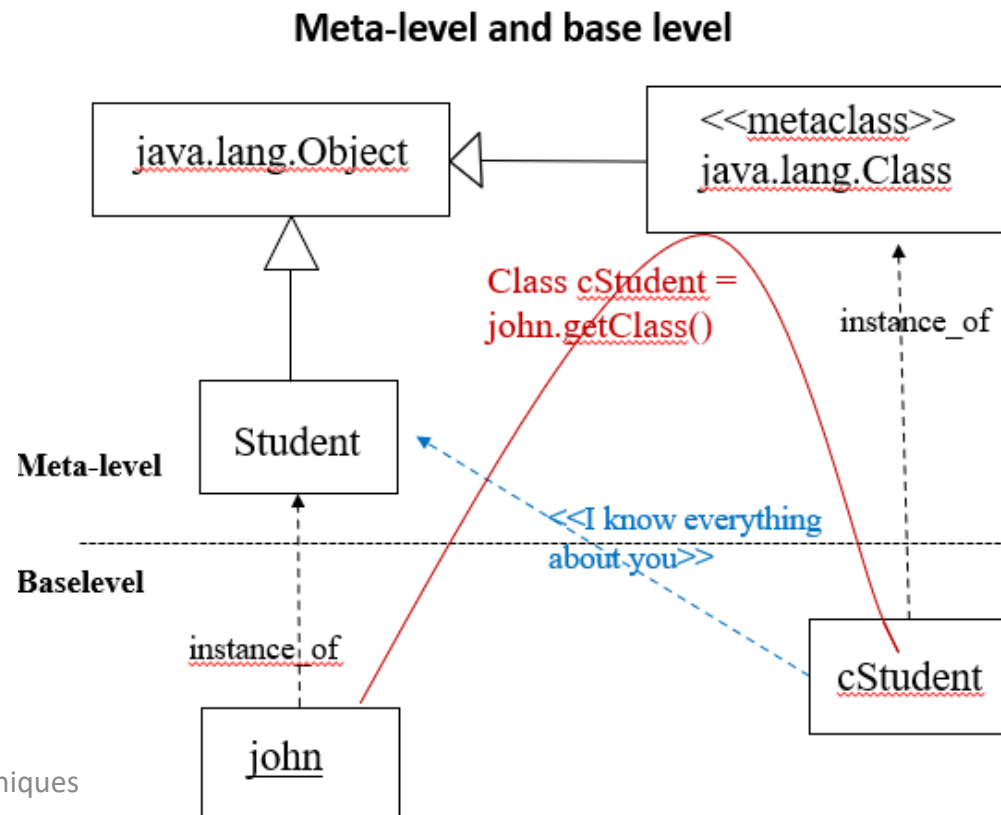
- Discover and use while a program is running
- Type information is represented at runtime as a **Class** object

You must first get
a reference to
the
appropriate Class
object



java.lang.Class - Overview

- Stores metadata for the class itself
 - Package and superclass, interfaces implemented
 - Details of the constructors, fields, and methods defined by the class



java.lang.Class - Building Class Objects

- **Java.lang.Class**

Getting a Class object

(1) Use method **getClass()** of the class **Object** if you have a reference to an object

```
Class c = "Alpha".getClass();  
  
Point p = new Point (2.1, 3.2);  
Class cp = p.getClass();  
  
Set<String> s = new HashSet<>();  
Class c = s.getClass();
```

(2) Use the static method **forName()** of the class **Class** when the type and name is available

```
package foo;  
public class Test {  
    public Test () {  
        System.out.println("Hello Test");  
    }  
  
    public static void main(String[] args)  
        throws Exception {  
        Class cls = Class.forName("foo.Test");  
        Test tst = (Test) cls.newInstance();  
    }  
}
```

java.lang.Class – Other Metadata Classes

- **Java.lang.Class** - other metadata classes

Metadata for a class

Metadata for

- constructor,
- fields,
- methods,
- etc.

```
class Class {
    Constructor[] getConstructors();
    Field getDeclaredField(String name);
    Field[] getDeclaredFields();
    Method[] getDeclaredMethods();
    ...
}

...

class Field {
    Class getType();
    ...
}

class Method {
    Class[] getParameterTypes();
    Class getReturnType();
    ...
}
```

Class Example

```
public class UserAccount {
    private int id;
    private String username;
    private String email;
    private boolean active;
    private double balance;
    public UserAccount() {}
    public UserAccount(int id, String username, String email, boolean active, double balance) {
        this.id = id;
        this.username = username;
        this.email = email;
        this.active = active;
        this.balance = balance;
    }
    public void activate() { this.active = true; }
    public void deactivate() { this.active = false; }
    public double applyDiscount(double percent) { return balance - (balance * percent / 100); }
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public boolean isActive() { return active; }
    public void setActive(boolean active) { this.active = active; }
    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; }
}
```

Reflection with Methods

- Retrieve all methods

```
UserAccount userAccount = new UserAccount(10, "user1", "user1@yahoo.com", true, 10.0);  
Class classObject = userAccount.getClass();  
Method[] methods = classObject.getDeclaredMethods();
```

- Retrieve a specific method if you know the details

```
Method m = classObject.getMethod("applyDiscount", new Class[]{double.class});
```

- Dynamically invoke a method on an object at runtime without specifying which method at compile time

```
m.invoke(userAccount, new Object[]{25.0})
```

- Get a method's parameter types

```
Class[] parameterTypes = m.getParameterTypes();
```

- Get a method's return type

```
Class returnType = method.getReturnType();
```

Reflection with Constructors

- ***java.lang.Reflect.Constructor***
 - Java Reflection API defines a meta class for dealing with constructors
 - represents Java constructors
- Interface to Constructor is like the interface to Method, except it supports a *newInstance* method instead of *invoke*
- Get a specific constructor if you know the details and create a new instance of the class

```
Constructor c = classObject.getConstructor(int.class, String.class, String.class, boolean.class, double.class);  
UserAccount userAccount1 = (UserAccount) c.newInstance(new Object[]{10, "user2", "user2@yahoo.com", true, 30.00});
```

Reflection with Fields

- Get the fields of a class

```
Field[] fields = classObject.getDeclaredFields();
```

- Access the value of a field

```
String username = (String) field.get(userAccount);
```

- Set the value of a field

```
Field field = classObject.getDeclaredField("username");  
field.setAccessible(true); // Disables all JVM runtime access checks to field  
field.set(userAccount, "user updated");
```

- Reflection with modifiers

```
public static Field[] getInstanceVariables(Class cls) {  
    List accum = new LinkedList();  
    while (cls != null) {  
        Field[] fields = cls.getDeclaredFields();  
        for (int i=0; i<fields.length; i++) {  
            if (!Modifier.isStatic(fields[i].getModifiers())) accum.add(fields[i]);  
        }  
        cls = cls.getSuperclass();  
    }  
    Field[] retvalue = new Field[accum.size()];  
    return (Field[]) accum.toArray(retvalue);  
}
```

Querying for fields can be disabled in the Java security manager.

Reflection with fields is useful for deserialization.

Reflection with Arrays

- ***java.lang.reflect.Array***

- Class for performing reflective operations on all array objects
- Getting the length of an array

```
int length = Array.getLength(obj);
```

- Get the i^{th} element of an array

```
Array.get(obj, i)
```


Applications of Reflection

- **Junit**

- Test methods are identified by an annotation - reflection is used to assign behavior to appropriate annotation at run time
- After finding those methods
 - using @Test, @BeforeTest, @AfterTest, they are invoked using reflection
- Naming conventions of methods are used to infer semantics

- **Auto-completion in a text editor**

- Java editors and IDEs provide auto-completion.
- The pop-up menu is populated by using Java reflection

Applications of Reflection

- **Spring**

- Uses reflection to create an object for each bean
- The object's type is specified by the class attribute

```
<bean id="someID" class = "com... .DomainClass"  
      <property name="someField" value="someValue" />  
</bean>
```

- When Spring processes this <bean> element will use to instantiate the corresponding class object

```
Class.forName("com... .DomainClass")
```

- After an object is constructed, each property is examined

```
obj.setXXX(value)
```

- By default, the object is created with its default constructor